

Build a Vulnerability Finder Agent

A week-long project for Python developers stepping into AI agent development — no prior LLM experience required.

DURATION	STACK	PREREQUISITES	OUTPUT	COST
5 days	Python · LangGraph · Groq	Python, no LLM exp.	CLI + HTML report	\$0 — all free-tier

Overview

What you're building

You'll spend the week building a security agent that takes a code repository — either a GitHub URL or a local folder — scans it for known vulnerabilities in the code itself, checks its dependencies against public CVE databases, and produces a formatted HTML report. The interesting part isn't the scanning — tools like `bandit` and `pip-audit` already do that. The interesting part is the *agent*: a graph of nodes that reason about input, route to the right tools, enrich raw findings with real CVE data, and hand off to a language model for the final report.

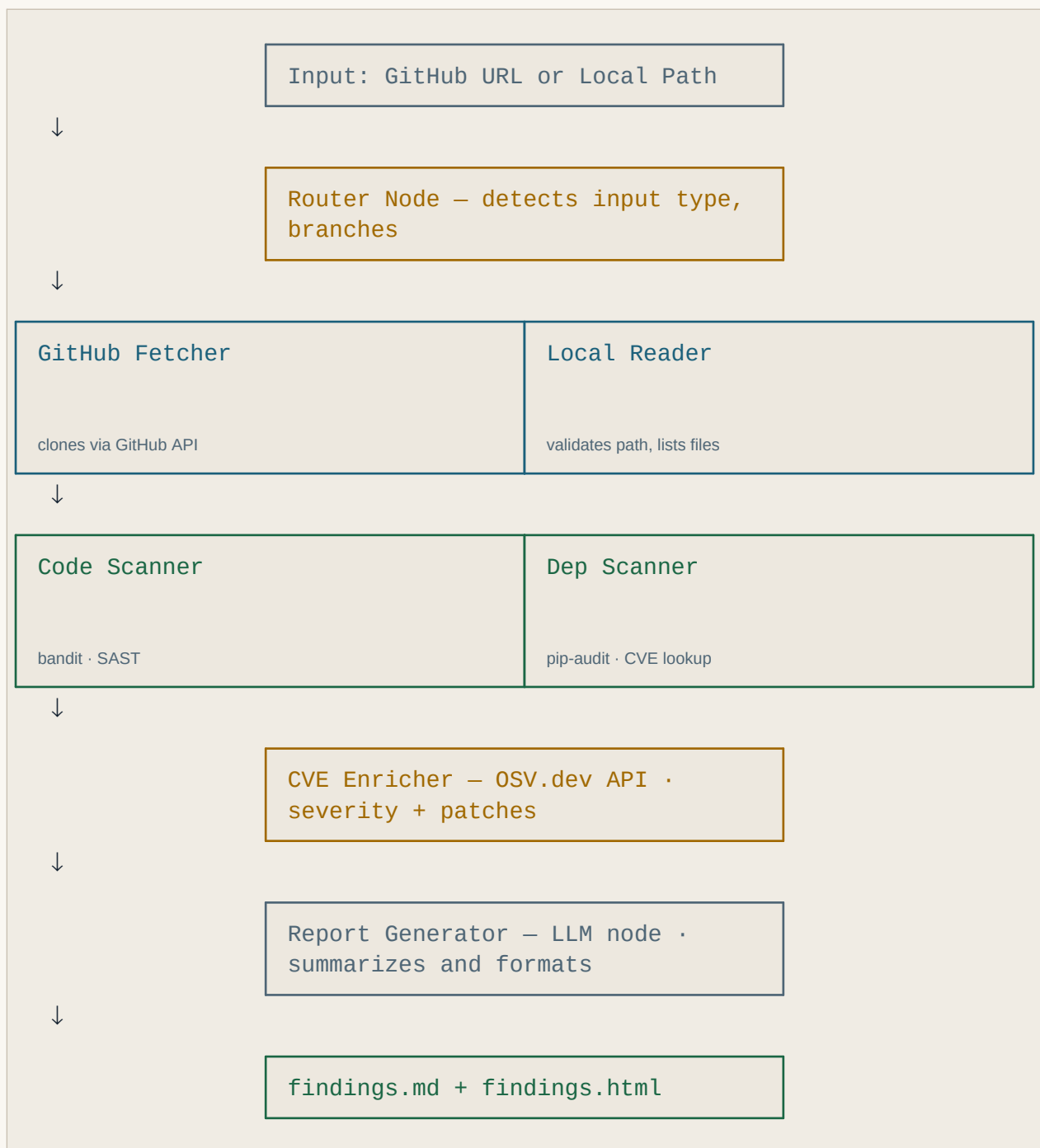
This is different from writing a script that runs those tools in sequence. LangGraph gives you state that accumulates across steps, conditional edges that branch based on what each node returns, and a model that can interpret results rather than just format them. By the end of Friday, you'll point it at an intentionally vulnerable Python repo and watch it produce a report you'd actually want to read.

Scope: the agent covers Python codebases only. Web and network scanning are explicitly out of scope. This keeps the project achievable and lets you go deep rather than wide.

Architecture

How the graph is structured

The agent is a directed graph with seven nodes. The router decides which fetcher to call based on whether the input is a URL or a path. Two scanner nodes run against the fetched code — one for static analysis, one for dependency checking. Their combined findings flow into a CVE enricher that pulls severity and patch data from the OSV database, then into a report generator backed by an LLM.



Schedule

Day-by-day breakdown

1 LangGraph Foundations + Project Setup

Understand the framework before touching the project. Don't skip this — the rest of the week depends on it.

- › Work through the official LangGraph quickstart. Focus on StateGraph, what a node is, what an edge is, and why state is typed.
- › Build a three-node 'hello world' graph: start → greet → end. Get it running, break it, fix it. The point is to feel the API.
- › Set up the project: requirements.txt, .env, folder structure, a README that explains what the project does.
- › Install and run the scanner tools manually — bandit -r ./sample, pip-audit — before writing a single line of agent code. Read the output. Know what you're wrapping.
- › Create a free Groq account and confirm you can make an API call from Python. This is your LLM for the week.

END-OF-DAY

You can draw your graph on paper and explain what 'state' means in LangGraph. The scanner tools run locally.

2 Tool Development

Build and test each tool in isolation. The agent is only as good as the tools it calls.

- › Write the GitHub fetcher tool: accepts a public repo URL, clones it to a temp directory using subprocess, and returns the local path. Handle malformed URLs and missing repos.
- › Write the local directory reader: validates the path, walks the file tree, returns a summary of languages detected by extension.
- › Wrap bandit as a tool: run with -f json, parse output, return a clean list of findings with file, line, severity, and issue type.
- › Wrap pip-audit similarly: parse JSON output, return vulnerable packages with current version and first safe version if one exists.
- › Write a quick sanity check for each tool — not a full test suite, just enough to confirm they return what you expect.

END-OF-DAY

Each tool works standalone. Call them from a Python script and get structured, predictable output every time.

3 Graph Wiring + State Design

Connect your tools into a real graph. This is where it starts feeling like an agent.

- › Design the VulnScanState TypedDict. Think through every field: input, fetched path, code findings, dep findings, CVE data, final report. The state schema is the contract between nodes.
- › Implement the router node: inspect the input string, decide whether it's a URL or a path, return the appropriate edge. This is your first conditional edge.
- › Wire the two fetcher nodes and connect them to a single merge point before the scanners. Both should populate the same repo_path field in state.
- › Add the scanner nodes. Both can run sequentially — parallel isn't needed and it's harder to debug.
- › Add a placeholder report generator that prints raw findings, and do an end-to-end test on a small real repo.

END-OF-DAY End-to-end run produces raw findings in state. Data flows through the full graph without errors.

4 CVE Enrichment + Report Generation

Turn raw findings into a report someone would actually want to read.

- › Implement the CVE enricher node: for each dependency finding, hit the OSV.dev API (api.osv.dev/v1/query) to fetch CVSS severity, affected versions, and patches. No API key needed.
- › Add severity classification: Critical, High, Medium, Low. Use CVSS scores from OSV where available, fall back to bandit's severity field for code findings.
- › Build the report generator node: pass all enriched findings to the LLM asking it to summarise the most important issues, group by severity, and explain why each one matters.
- › Convert the Markdown output to HTML using the markdown Python library. Write the file with a timestamp in the filename.
- › Add a basic stylesheet to the HTML report so it's readable in a browser without extra setup.

END-OF-DAY Pointing the agent at a real repo produces a findings.html that's readable, severity-sorted, and includes CVE references for dependency issues.

5

Hardening, Testing, Demo

Make it handle the real world. Test against repos designed to break tools like this.

- › Test against fportantier/pygoat — an intentionally vulnerable Django app. Run the agent, read the report, check whether findings are accurate and summaries make sense.
- › Test against a clean repo with no known issues. Confirm the agent produces a report that says so clearly, rather than returning nothing or erroring.
- › Handle the obvious failure modes: private GitHub repo, empty directory, bandit finds nothing (a valid result, not an error), OSV.dev is slow (log and continue without enrichment).
- › Build a clean CLI with argparse: `python agent.py --target --output ./reports`. Print a one-line status as each node completes.
- › Write a README covering what it does, setup, how to get a Groq key, how to run it, and example output. This is part of the deliverable.

END-OF-DAY Live demo: take a GitHub URL you've never used, run the agent, show the HTML report in the browser. The whole run takes under 90 seconds.

Setup

Tools and stack

Everything here is free — no credit card, no paid tier required. Get a Groq account on Day 1; the rest installs with pip.

TOOL	WHAT IT DOES	HOW TO GET IT
<code>langgraph</code> + <code>langchain-core</code>	The agent framework. Defines your graph, state, nodes, and edges.	<code>pip install langgraph langchain-core</code>
<code>groq</code> Free Llama 3 inference	The LLM powering the report generator node. Fast and free with a generous rate limit.	<code>pip install groq</code> <code>groq.com/keys</code>
<code>bandit</code>	Python static analysis. Finds hardcoded secrets, injection risks, weak crypto, and ~100 other patterns.	<code>pip install bandit</code>
<code>pip-audit</code>	Checks your requirements.txt against the OSV and PyPI Advisory databases.	<code>pip install pip-audit</code>
<code>OSV.dev API</code> <code>api.osv.dev/v1/query</code>	Google's open-source vulnerability database. Returns CVSS scores, affected versions, and patch info. No key needed.	HTTP · no auth required
<code>GitHub REST API</code>	Validates a repo URL and fetches metadata before cloning. No key for public repos.	No key for public repos
<code>markdown</code>	Converts the LLM's Markdown output to HTML for the final report file.	<code>pip install markdown</code>

Outcomes

What you'll come away with

This project covers four distinct areas. You won't be an expert in any of them by Friday, but you'll have real, working experience with each — which is different from having read about them.

LangGraph & Agent Fundamentals	Tool Use & API Integration	· Wrapping subprocess
Security Domain Knowledge	Software Engineering Practice	· Designing

Heads up

Where people usually get stuck

These are the places where interns on similar projects have lost time. None of them are blockers if you know they're coming.

WHAT TRIPS PEOPLE UP	LIKELIHOOD	HOW TO HANDLE IT
LangGraph's state model doesn't click on Day 1 — specifically, that nodes return partial state updates rather than mutating state directly.	Medium	Re-read the 'how state updates work' docs section. Draw the flow on paper. A 20-minute pairing session here saves 2 hours of confusion.
Bandit produces a lot of false positives on some codebases. Easy to assume the findings are wrong rather than learning to read the confidence levels.	Medium	Run bandit manually on Day 1 and read the raw output before writing any code around it. Understanding what it flags makes it much easier to parse programmatically.
The OSV.dev API can be slow on large dependency lists. Calling it synchronously in a loop may time the enricher node out.	Low	Cap enrichment at the top 20 findings. The goal is a working agent, not exhaustive coverage of every package.
Scope creep: wanting to add web endpoint scanning, network scanning, or support for non-Python repos mid-week.	Low	Python repos, code + deps only. Write that down somewhere visible on Day 1. A scoped thing that works beats an ambitious thing that doesn't.